INDEX

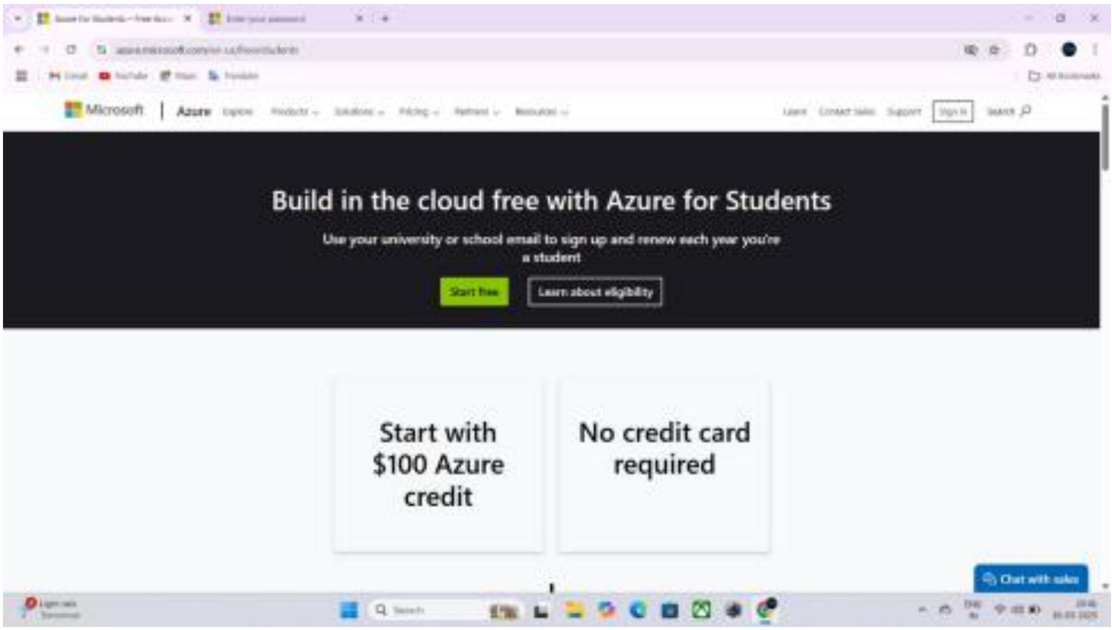
EXP NO: 1 DATE:	AZURE DEVOPS ENVIRONMENT SETUP

AIM: -

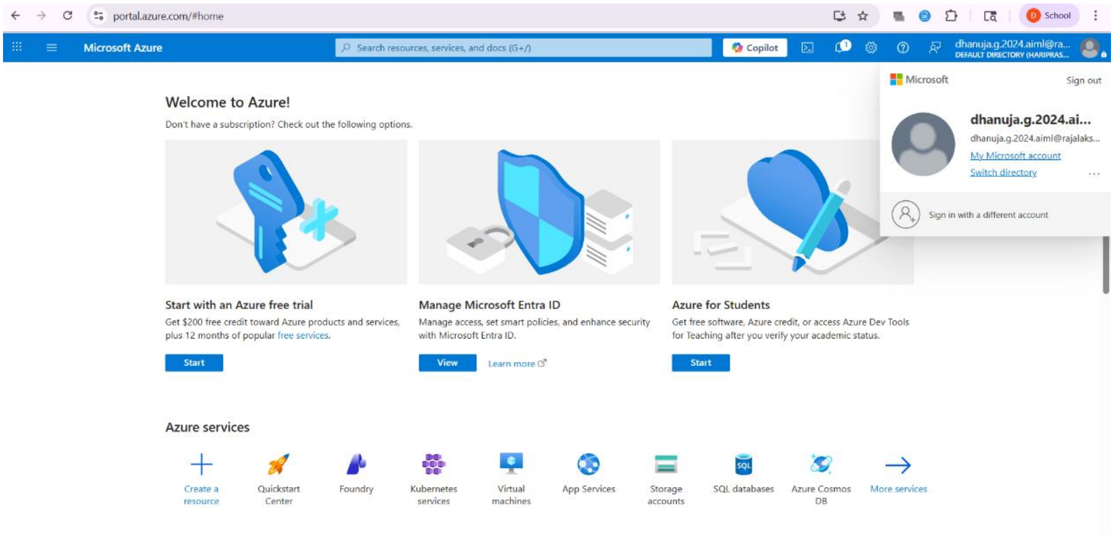
To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION: -

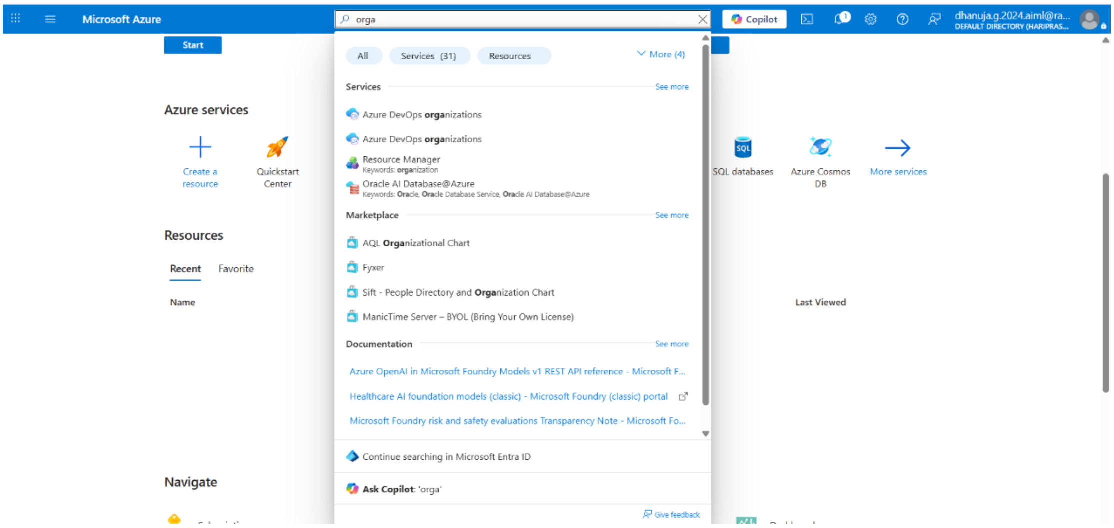
1. Open your web browser and go to the Azure website: https://azure.microsoft.com/en-us/get- started/azure-portal.
2. Sign in using your Microsoft account credentials.
3. If you don't have a Microsoft account, you can create one here: https://signup.live.com/?lic=1



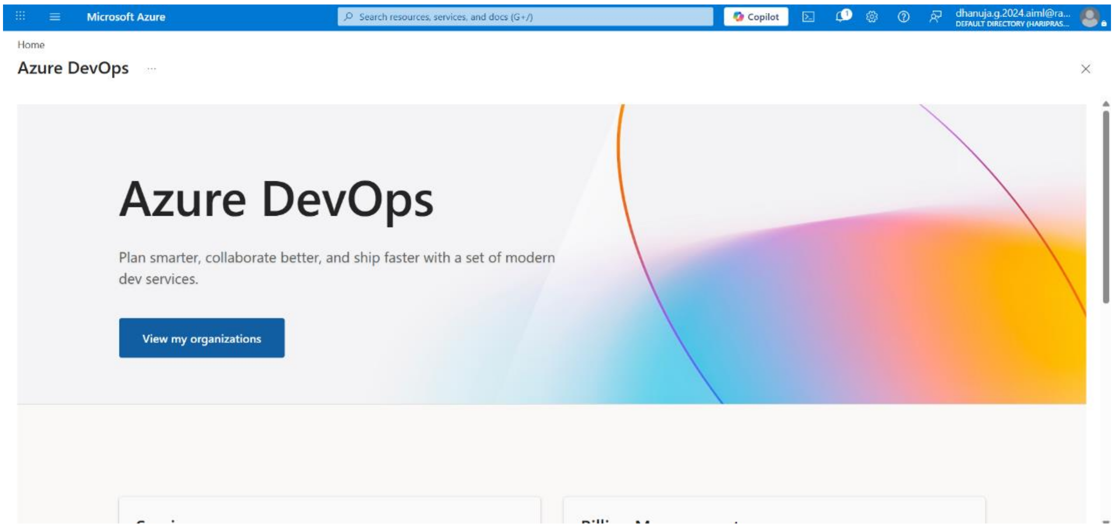
4. Azure home page



5. Open DevOps environment in the Azure platform by typing ***Azure DevOps Organizations*** in the search bar.



6. Click on the ***My Azure DevOps Organization*** link and create an organization and you should be taken to the Azure DevOps Organization Home page.



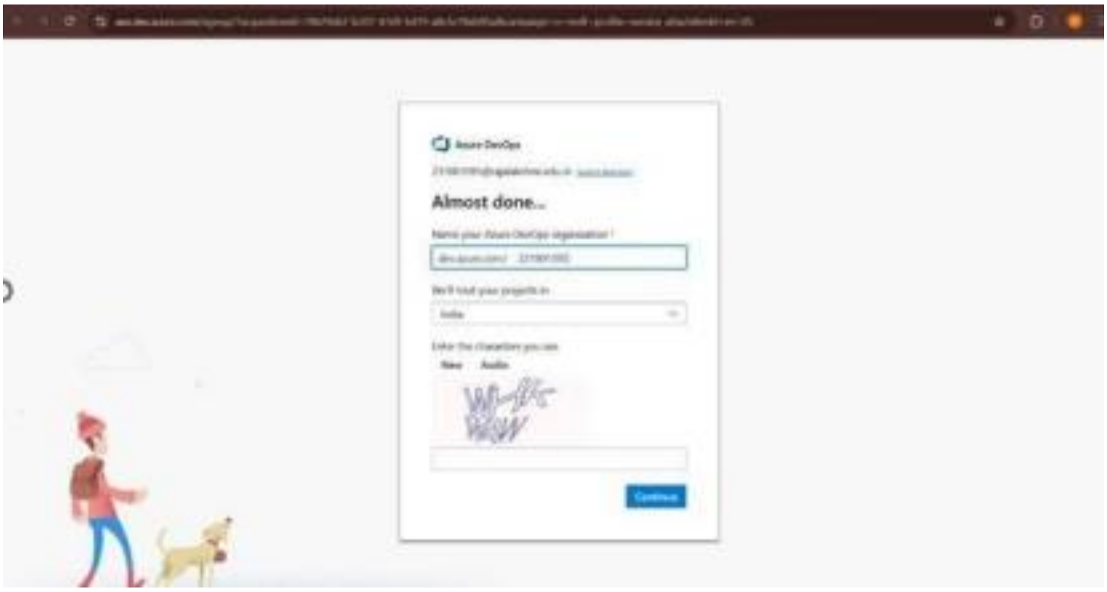
RESULT:

Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXP NO: 2 **DATE:**	**AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT**

AIM: To set up an Azure DevOps project for efficient collaboration and agile work management.

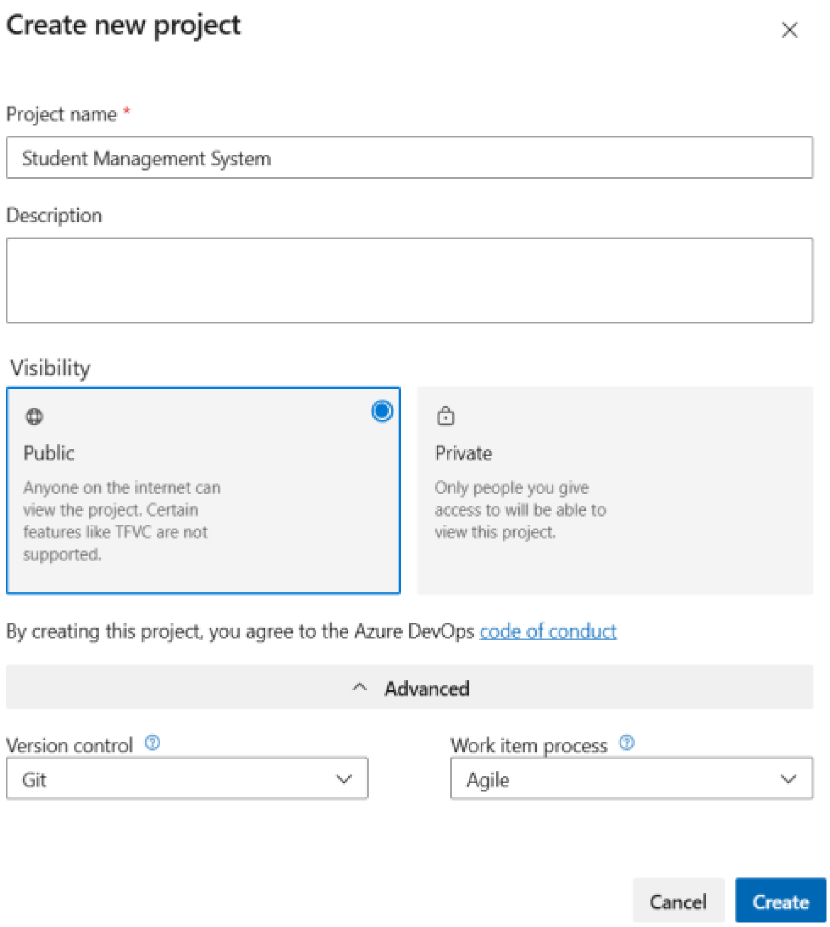
1. Create An Azure Account



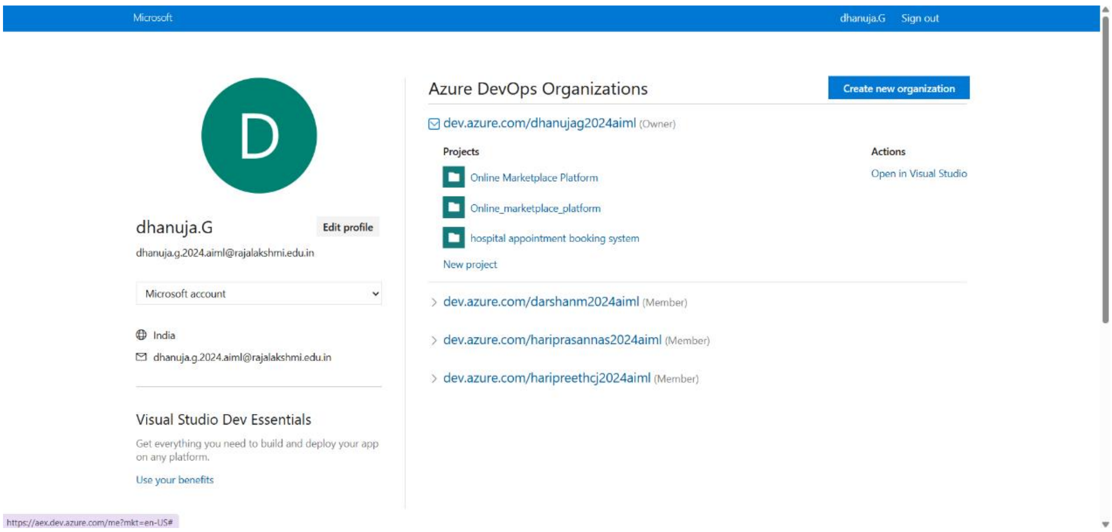
2. Create the First Project in Your Organization a) After the organization is set up, you'll need to create your first project. This is where you'll begin to manage code, pipelines, work items, and more. b) On the organization's Home page, click on the New Project button. c) Enter the project name, description, and visibility options:

    - **Name:** Choose a name for the project (e.g., LMS).
    - **Description:** Optionally, add a description to provide more context about the project.
    - **Visibility:** Choose whether you want the project to be Private (accessible only to those invited) or Public

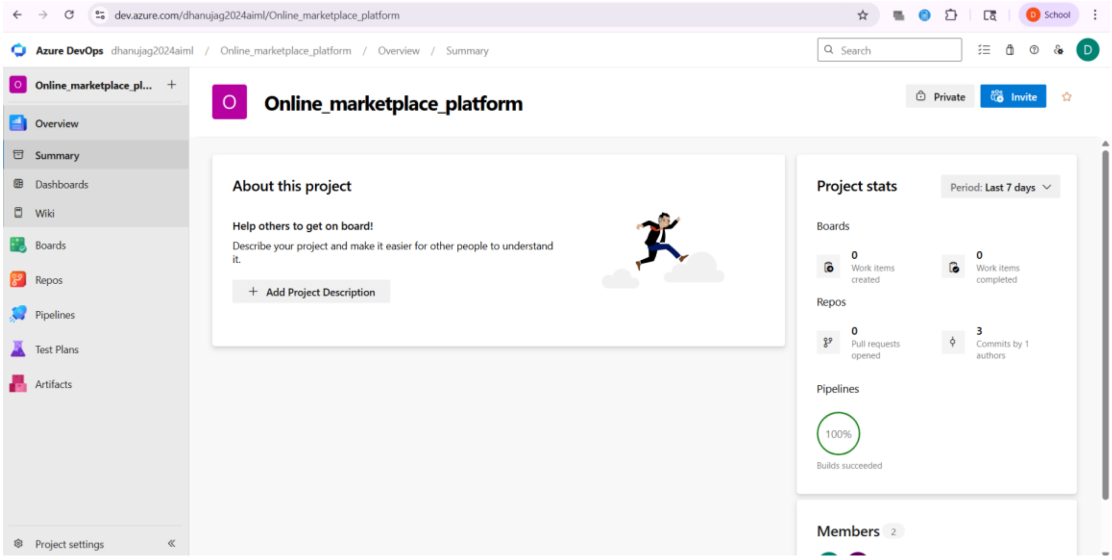
(accessible to anyone). d) Once you've filled out the details, click Create to set up your first project.



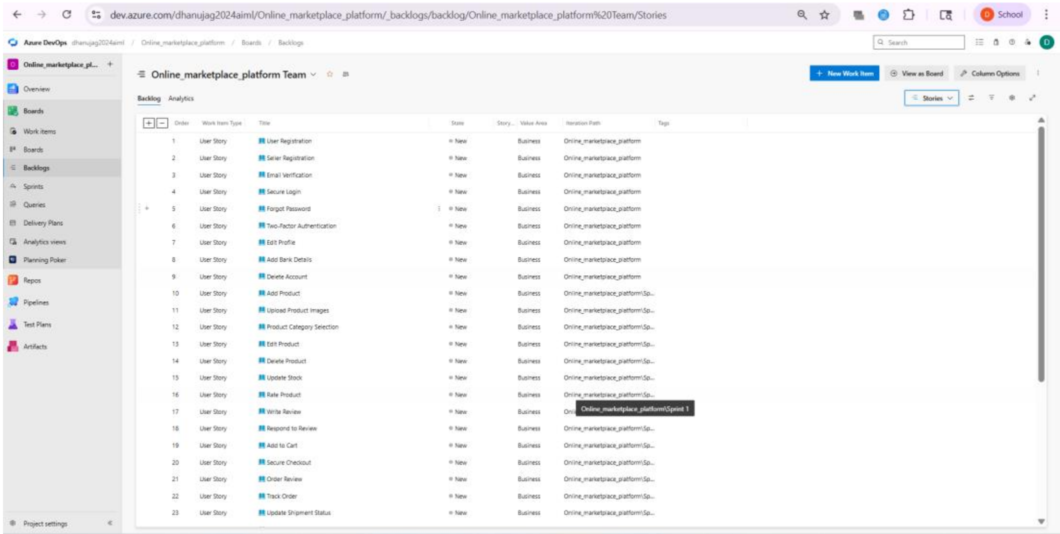
3. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.

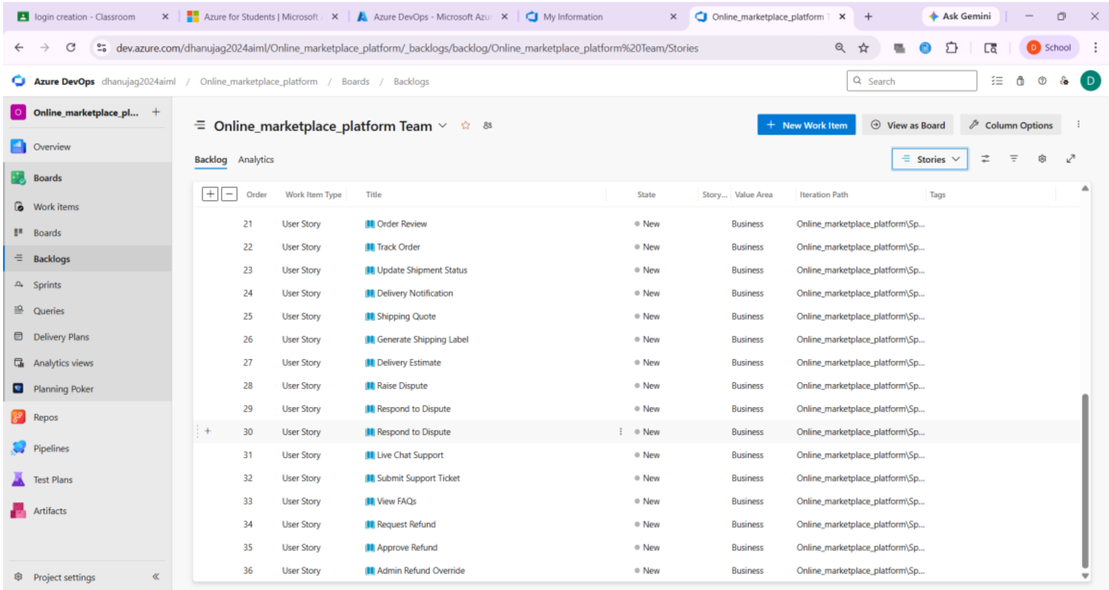


4. Project dashboard



5. To manage user stories: a. From the left-hand navigation menu, click on Boards. This will take you to the main Boards page, where you can manage work items, backlogs, and sprints. b. On the work items page, you'll see the option to Add a work item at the top. Alternatively, you can find a + button or Add New Work Item depending on the view you're in. From the Add a work item dropdown, select User Story. This will open a form to enter details for the new User Story.





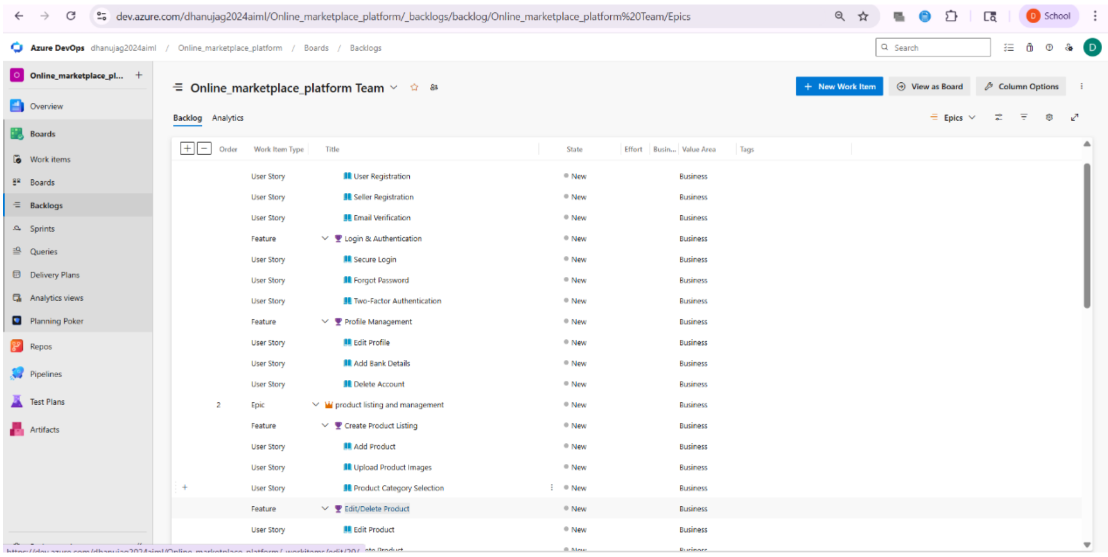
RESULT: Successfully created an Azure DevOps project with user story management and agile workflow setup.

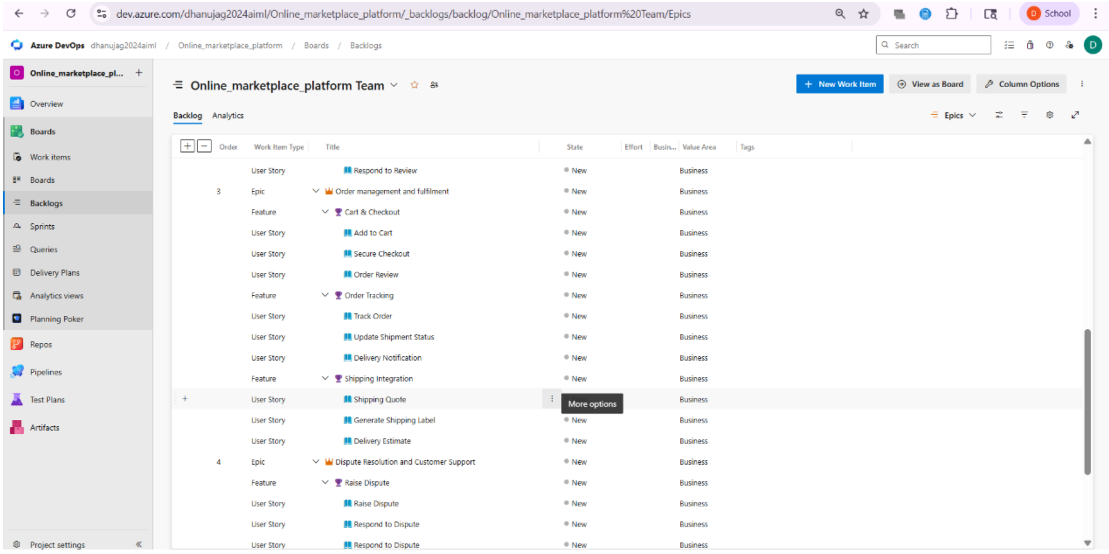
EXP NO: 3 **DATE:**	**SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING**

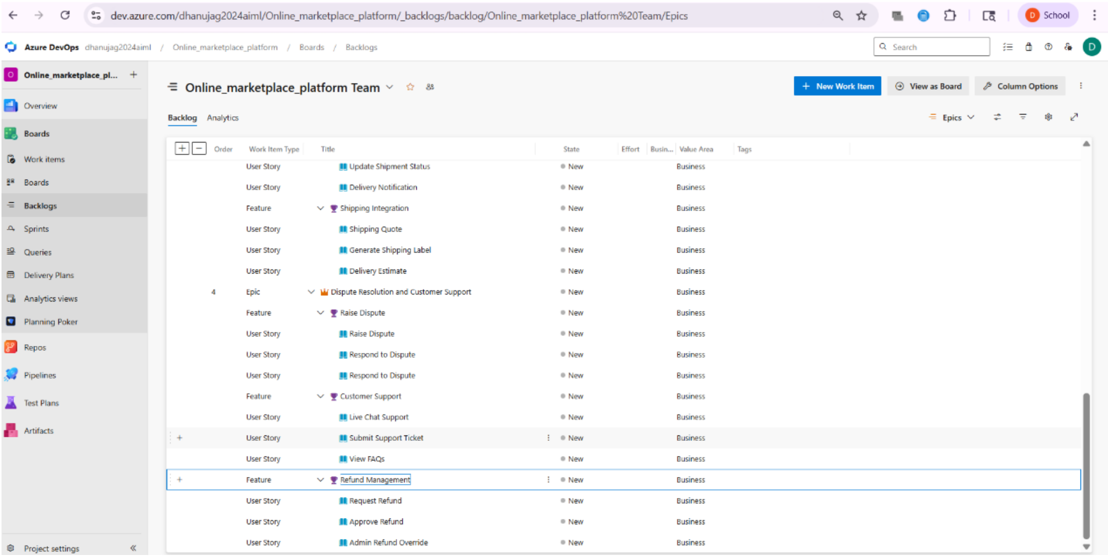
AIM: To learn about how to create epics, user story, features, backlogs for your assigned project.

Create Epic, Features, User Stories, Task

1. Fill in Epics
2. Fill in Features
3. Fill in User Story Details







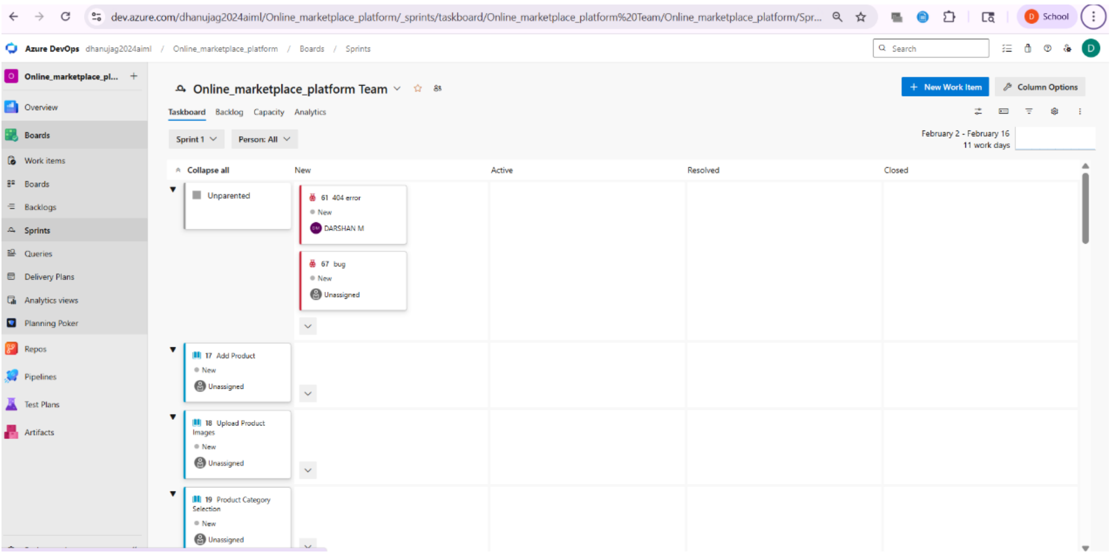
RESULT: Thus, the creation of epics, features, user story and task has been created successfully.

EXP NO: 4 DATE:	SPRINT PLANNING

AIM: To assign user story to specific sprint for the Student Management System Project.

Sprint Planning:

- Sprint 1
- Sprint 2
- Sprint 3

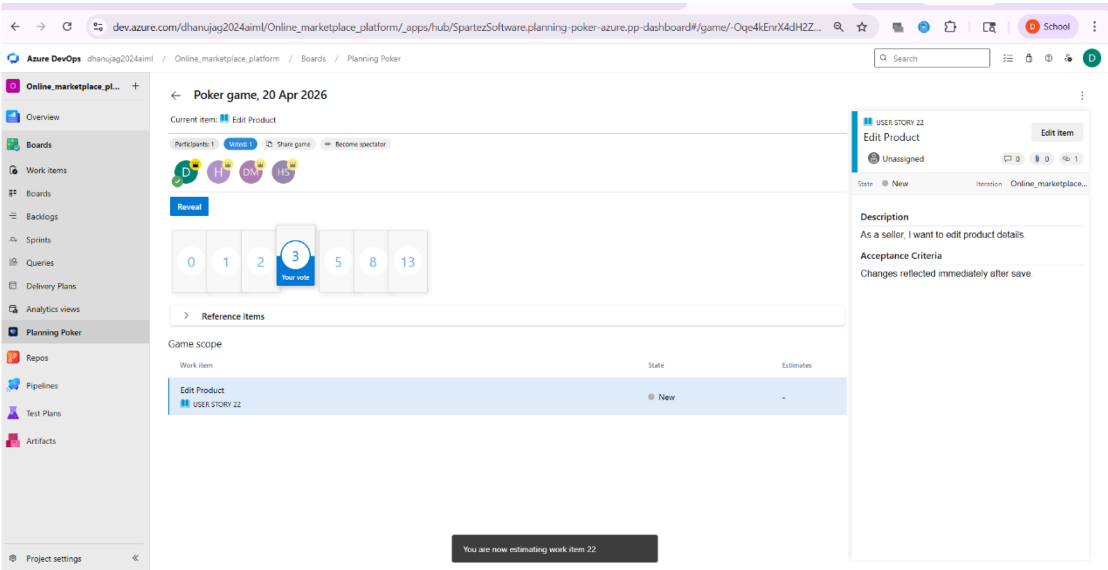


RESULT: The Sprints are created for the Student Management System.

EXP NO: 5 **DATE:**	**POKER ESTIMATION**

AIM: Create Poker Estimation for the user stories - Student Management System.

Poker Estimation:

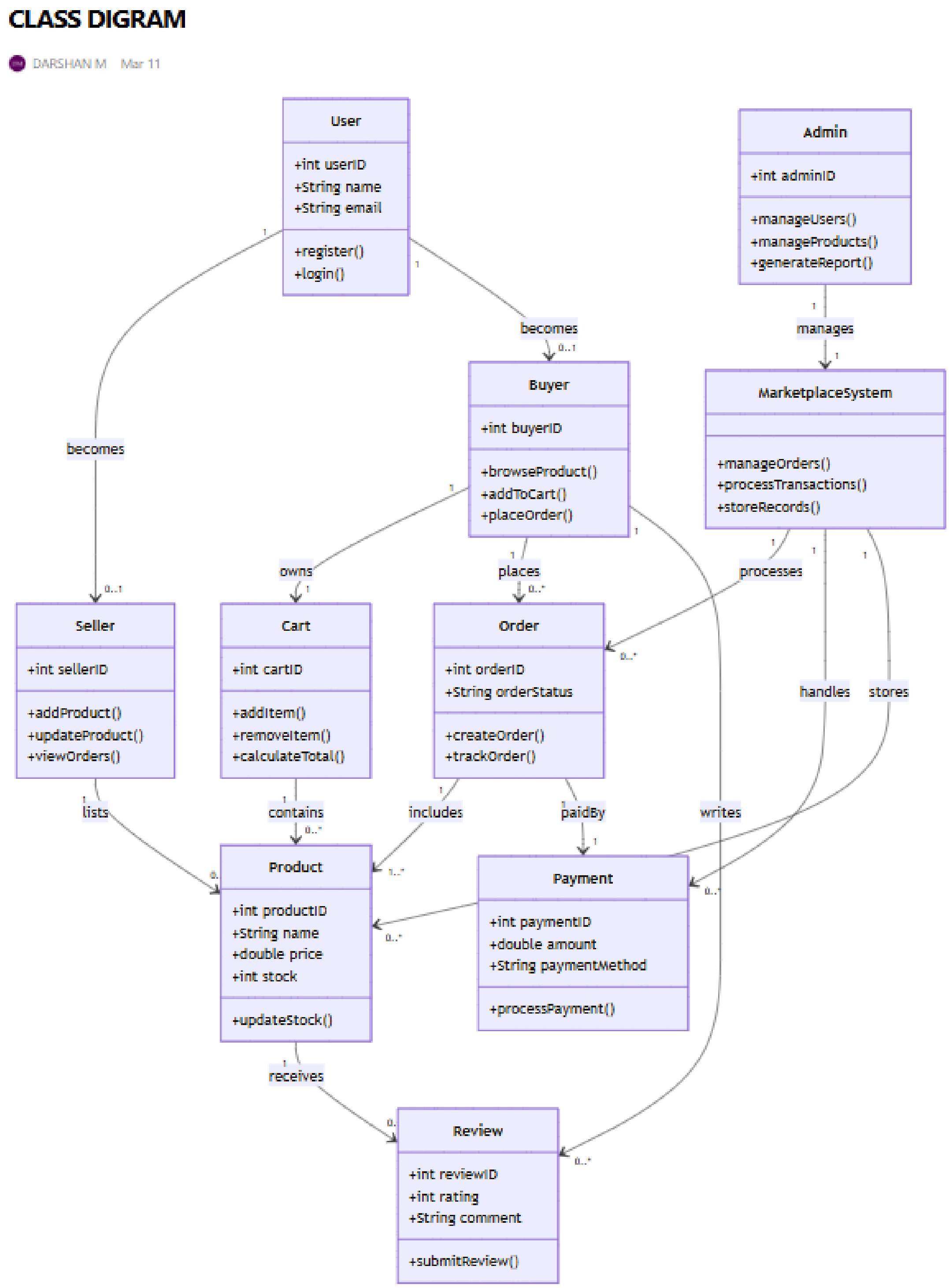


RESULT: The Estimation/Story Points is created for the project using Poker Estimation.

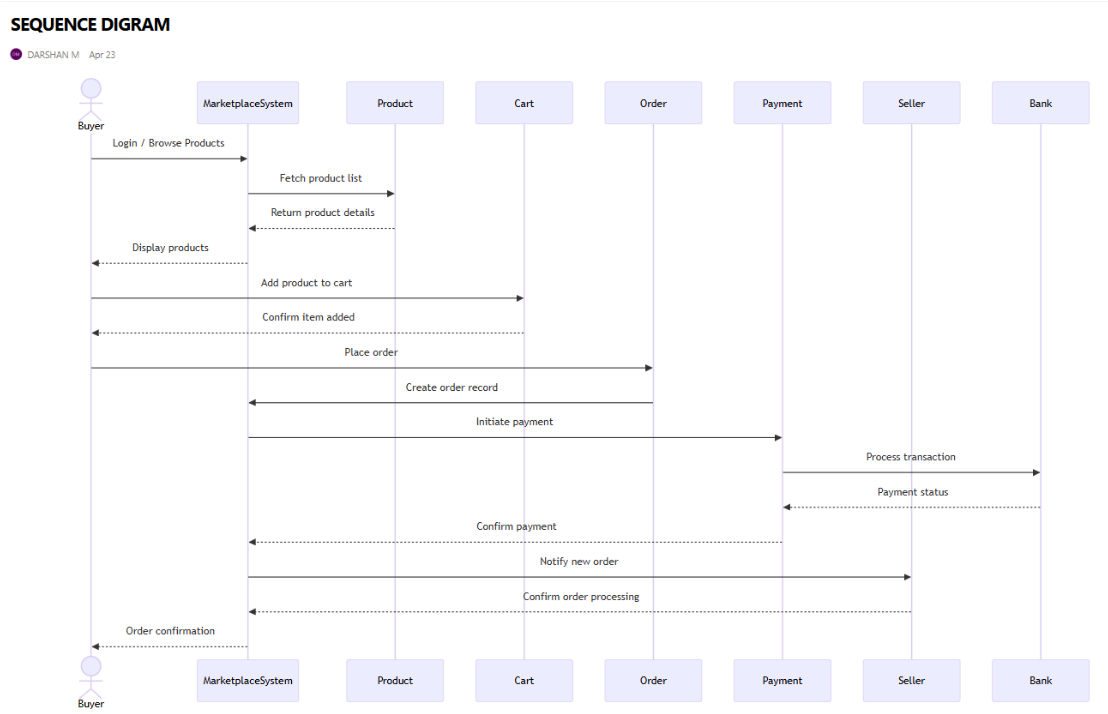
EXP NO: 6 **DATE:**	**DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE**

AIM: To Design a Class Diagram and Sequence Diagram for the given Project.

6A. CLASS DIAGRAM:



6B. SEQUENCE DIAGRAM:

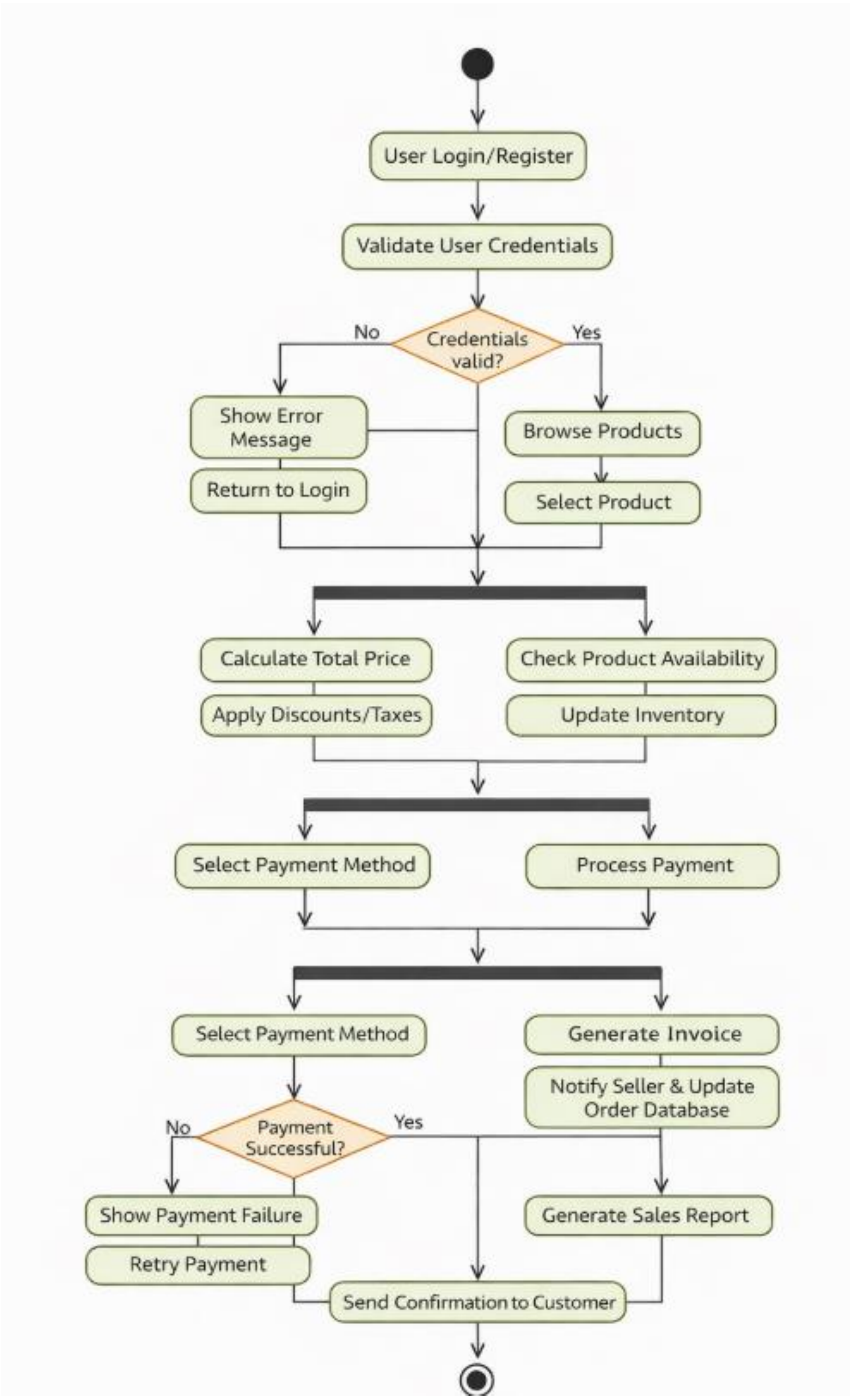


RESULT: The Class Diagram and Sequence Diagram is designed Successfully for given project.

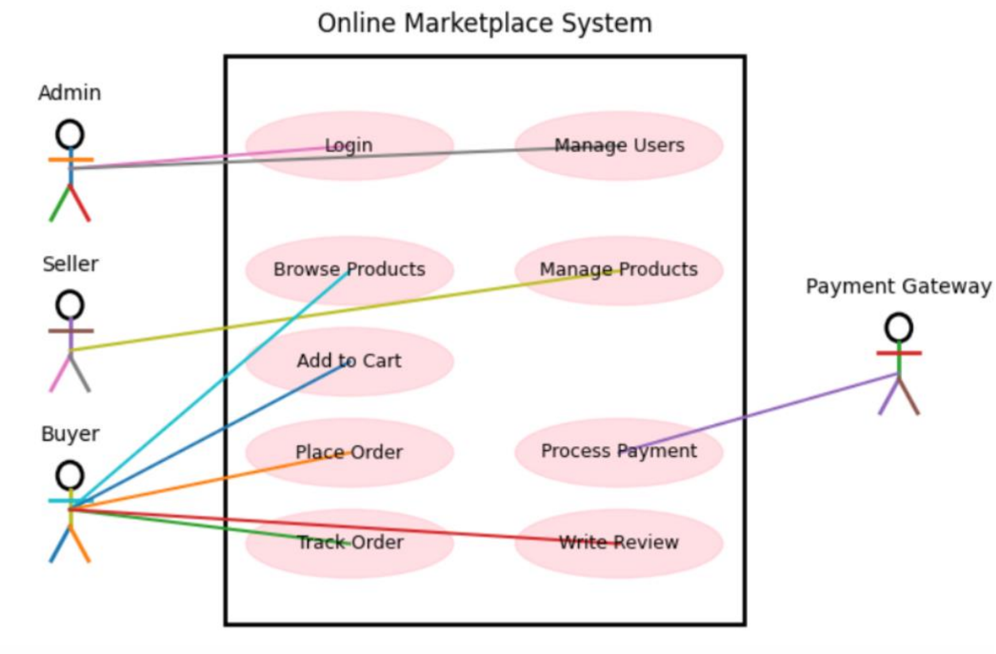
EXP NO: 7 DATE:	DESIGNING USE CASE DIAGRAMS AND ACTIVITY DIAGRAM FOR PROJECT ARCHITECTURE

AIM: To Design an activity Diagram and use case Diagram for the given Project.

7A. ACTIVITY DIAGRAM:



7B. USE CASE DIAGRAM:



RESULT: Thus activity and use case diagram has been designed successfully for Student Management System.

EXP NO: 8 DATE:	TESTING – TEST PLANS AND TEST CASES

AIM: Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case

Test Case Design Procedure

1. Understand Core Features of the Application

- User Signup & Login
- Student Enrollment in Courses
- Assignment Submission & Evaluation
- Viewing Grades and Attendance
- Admin Role Management and Report Generation

2. Define User Interactions

- Each test case simulates a real user behavior (e.g., logging in, submitting an assignment, viewing results).

3. Design Happy Path Test Cases

- Focused on validating that all core functionalities work correctly under normal conditions.
- Example: Student registers and logs in, enrolls in a course, submits assignment, and views grades.

4. Design Error Path Test Cases

- Simulate invalid inputs, system issues, or failed actions to ensure proper error handling.
- Example: Login with wrong credentials, submission without attachment, unauthorized access to admin panel.

5. Break Down Steps and Expected Results

- Each test case includes a clear sequence of actions and expected results.
- Ensures both manual testers and automation tools can follow the process easily.

6. Use Clear Naming and IDs

- Test cases are uniquely identified (e.g., TC01 – Student Login Success, TC12 – Invalid Assignment Submission).
- Facilitates easy mapping to features and tracking in Azure DevOps.

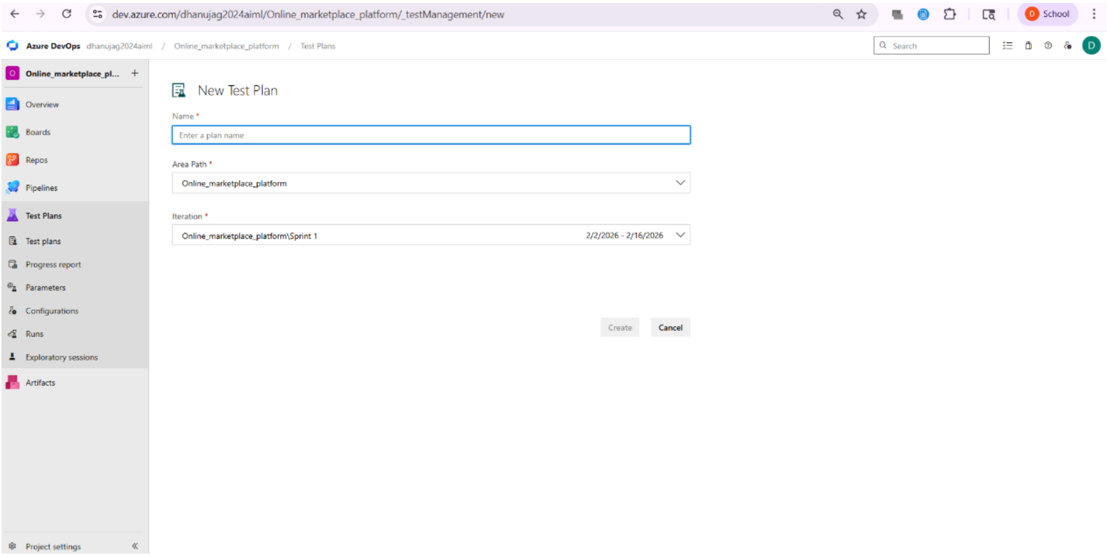
7. Separate Test Suites

- Grouped by functionality such as:
    - Login and Registration
    - Course Enrollment
    - Assignment Submission
    - Report Generation
    - Admin Functions
- Improves organization and enables focused execution in Azure DevOps.

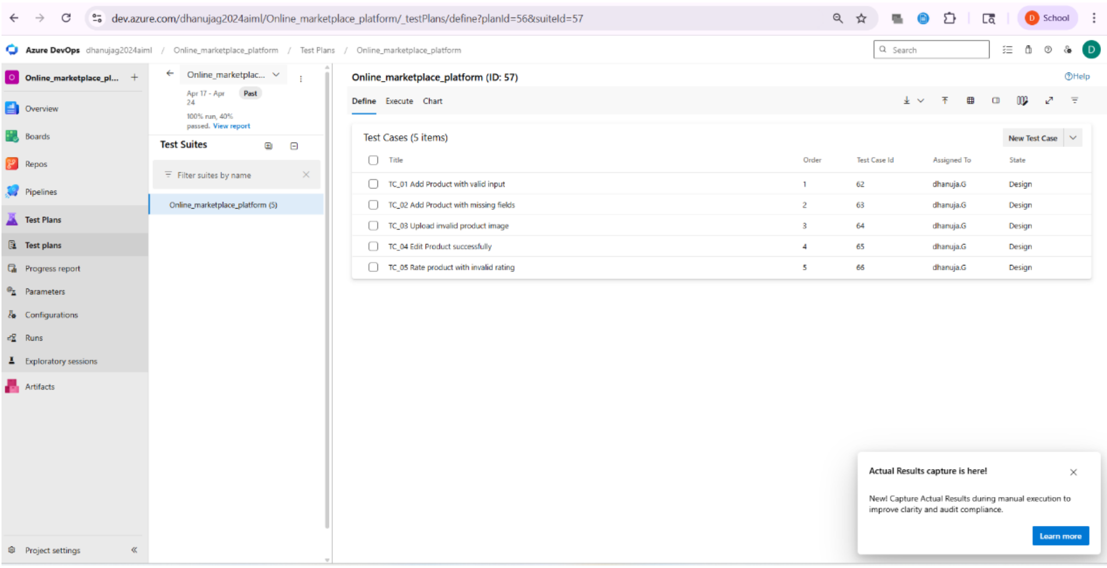
8. Prioritize and Review

- High-priority assigned to critical workflows like login, course access, and grading.
- Reviewed for completeness, accuracy, and alignment with user stories and feature definitions.

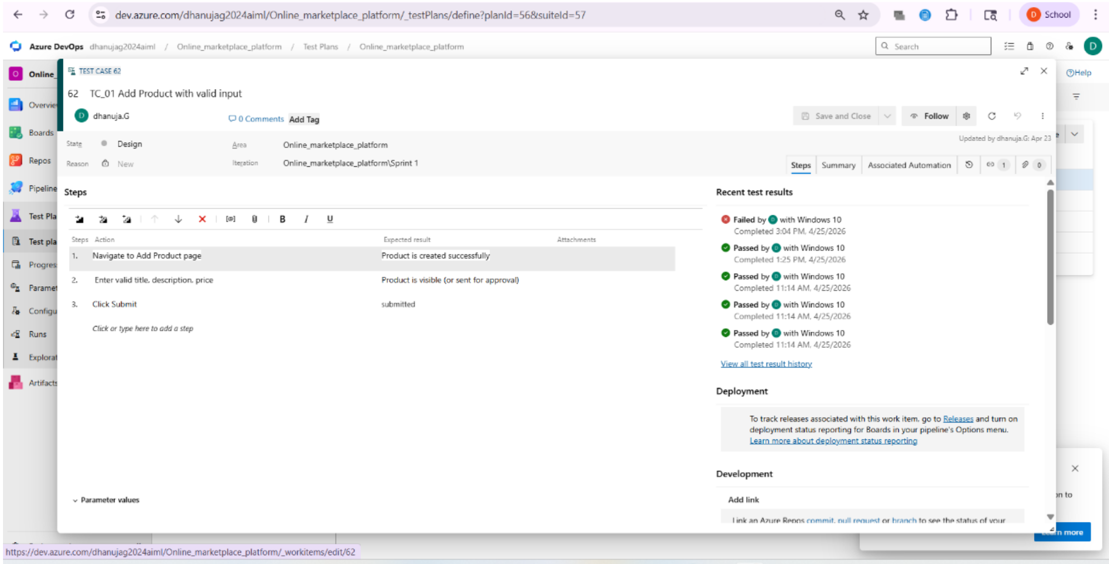
1. New test plan



2. Test suite



3.Test case



Give two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Online Marketplace Platform– Test Plans

USER STORIES:

- As a student, I want to sign up and log in securely so I can access academic services. (ID: 101)
- As a student, I need to enroll in courses to participate in classes. (ID: 102)
- As a student, I should be able to submit assignments online for evaluation. (ID: 103)
- As a student, I want to view grades and attendance in one place. (ID: 104)
- As an admin, I want to manage users and generate performance reports. (ID: 105)

Test Suites:

TC_01: Student Login (Valid Credentials)

User Story ID: 101
Steps:

1. **Open login page**
2. **Enter valid student email and password**
3. **Click login button**
4. **Verify dashboard is displayed**

Expected Result: Student logs in successfully and is redirected to dashboard
Type: Happy Path

TC_02: Student Login (Invalid Credentials)

User Story ID: 101
Steps:

1. **Open login page**
2. **Enter incorrect email or password**
3. **Click login button**
4. **Verify error message is displayed**

Expected Result: Error message "Invalid username or password" is shown
Type: Error Path

TC_03: Course Enrollment (Valid)

User Story ID: 102
Steps:

1. **Login to the system**
2. **Navigate to course list**
3. **Select available course**
4. **Click enroll**

Expected Result: Enrollment confirmation is displayed
Type: Happy Path

TC_04: Course Enrollment (Course Full)

User Story ID: 102
Steps:

1. **Login to the system**
2. **Select a course with no available seats**
3. **Click enroll**
4. **Verify system response**

Expected Result: Error message "Course full" is displayed
Type: Error Path

TC_05: Assignment Submission (Valid)

User Story ID: 103
Steps:

1. **Login to the system**
2. **Navigate to assignments**
3. **Upload assignment file**
4. **Click submit**

Expected Result: Assignment submitted successfully
Type: Happy Path

TC_06: Assignment Submission (No File)

User Story ID: 103
Steps:

1. **Open assignment submission page**
2. **Do not upload any file**

3. **Click submit**
4. **Verify error message**

Expected Result: Message "Please upload a file before submitting"
Type: Error Path

TC_07: View Grades (Valid)

User Story ID: 104
Steps:

1. **Login to the system**
2. **Navigate to grades section**
3. **View grades**
4. **Verify data is displayed**

Expected Result: Grades are displayed correctly
Type: Happy Path

TC_08: View Grades (Network Failure)

User Story ID: 104
Steps:

1. **Disconnect internet connection**
2. **Navigate to grades section**
3. **Attempt to load data**
4. **Verify error message**

Expected Result: "Unable to fetch data" message is shown
Type: Error Path

TC_09: Admin Role Assignment (Valid)

User Story ID: 105
Steps:

1. **Login as admin**
2. **Navigate to user management**
3. **Select user and assign role**
4. **Save changes**

Expected Result: Role assigned successfully
Type: Happy Path

TC_10: Report Generation (No Data)

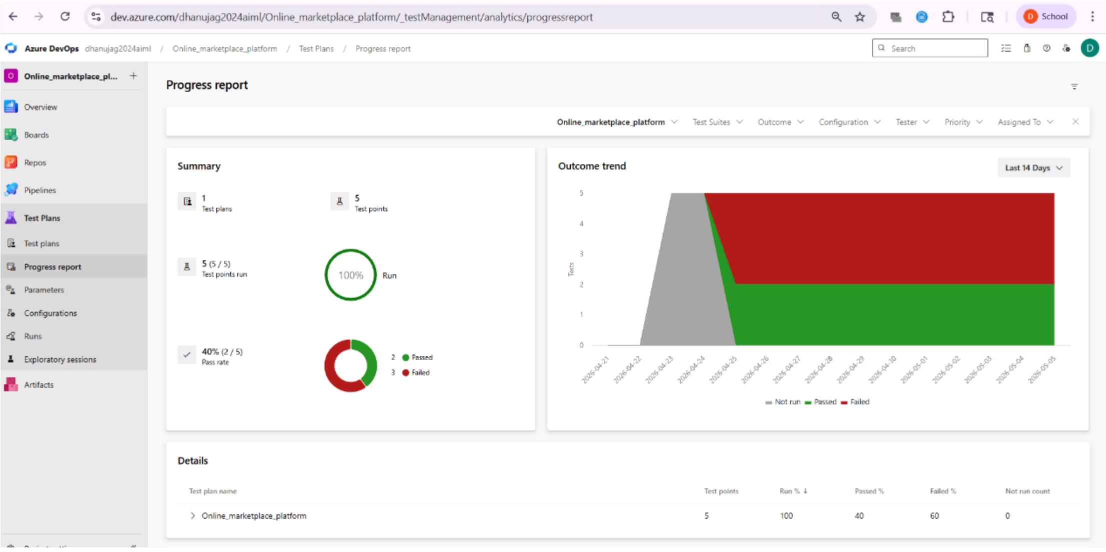
User Story ID: 105
Steps:

1. **Login as admin**
2. **Navigate to reports section**
3. **Select empty dataset/semester**
4. **Click generate report**

Expected Result: Message "No data available" is displayed
Type: Error Path

Test Results

8. Test report summary
9. Assigning bug to the developer and changing state
10. Progress report
11. Changing the test template



RESULT: The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path.

EXP NO: 9 DATE:	CI/CD PIPELINES IN AZURE

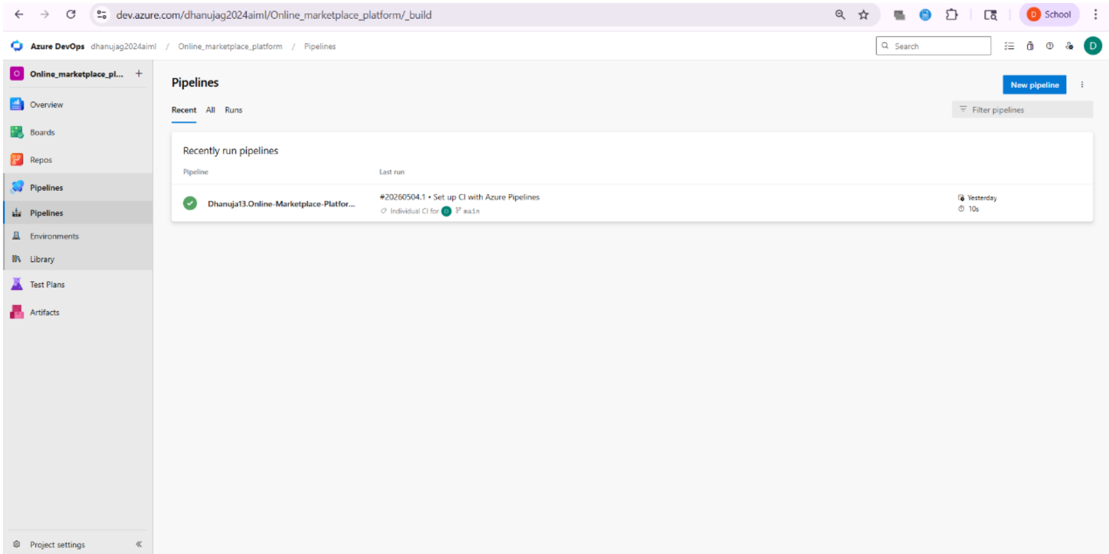
AIM: To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline in Azure DevOps for automating the build, testing, and deployment process of the Student Management System, ensuring faster delivery and improved software quality.

PROCEDURE:

Steps to Create and implement pipelines in Azure:

1. **Sign in to Azure DevOps and Navigate to Your Project** Log in to dev.azure.com, select your organization, and open the project where your Student Management System code resides.
2. **Connect a Code Repository (Azure Repos or GitHub)** Ensure your application code is stored in a Git-based repository such as Azure Repos or GitHub. This will be the source for triggering builds and deployments in your pipeline.
3. **Create a New Pipeline** Go to the Pipelines section on the left panel and click "Create Pipeline". Choose your source (e.g., Azure Repos Git or GitHub), and then select the repository containing your project code.
4. **Choose the Pipeline Configuration** You can select either the YAML-based pipeline (recommended for version control and automation) or the Classic Editor for a GUI-based setup. If using YAML, Azure DevOps will suggest a template or allow you to define your own.
5. **Define Build Stage (CI - Continuous Integration) from YAML file**
6. Install dependencies (e.g., npm install, dotnet restore)
7. Build the application (dotnet build, npm run build)
8. Run unit tests (dotnet test, npm test)

9. Publish build artifacts to be used in the release stage
10. **Save and Run the Pipeline for the First Time** Save the YAML or build definition and click "Run". Azure will fetch the latest code and execute the defined build and test stages.
11. **Configure Continuous Deployment (CD)** Navigate to the Releases tab under Pipelines and click "New Release Pipeline". Add an Artifact (from the build stage) and create a new Stage (e.g., Development, Production).
12. Configure the CD stage with deployment tasks such as deploying to Azure App Service, running database migrations or scripts, and restarting services using the Azure App Service Deploy task linked to your subscription and app details.
13. **Set Triggers and Approvals** Enable continuous deployment trigger so the release pipeline runs automatically after a successful build. For production environments, configure pre-deployment approvals to ensure manual verification before release.
14. **Monitor Pipelines and Manage Logs** View all pipeline runs under the Runs section. Check logs for build/test/deploy stages to debug any errors. You can also integrate email alerts or Microsoft Teams notifications for build failures.
15. **Review and Maintain Pipelines** Regularly update your pipeline tasks or YAML configurations as your application grows. Ensure pipeline runs are clean and artifacts are stored securely. Integrate quality gates and code coverage policies to maintain code quality.

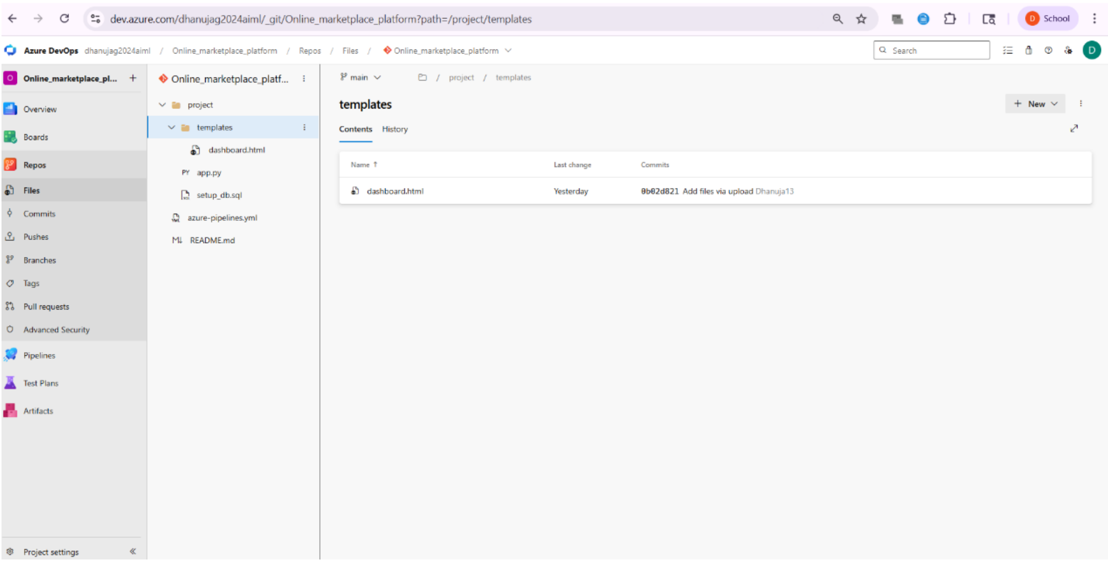


RESULT: Thus the pipelines for the given project " Online Marketplace Platform " has been executed successfully.

EXP NO: 10 DATE:	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

AIM: To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Student Management System project.

GitHub Project Structure



RESULT: The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.